UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FACULDADE DE COMPUTAÇÃO
BACHARELADO EM SISTEMAS DE INFORMAÇÃO

[nomes e matrículas dos integrantes do grupo]

SISTEMA DE BIBLIOTECA

APLICAÇÃO DOS CONCEITOS DE PROGRAMAÇÃO ORIENTADA A OBJETOS NO CONTROLE DE ACERVO, LEITORES E EMPRÉSTIMOS

Trabalho final apresentado à disciplina de Programação Orientada a Objetos, turma 1B141, do curso de Sistemas de Informação da Faculdade de Computação da Universidade Federal de Uberlândia, sob orientação do Prof. Roberto Caetano.

UBERLÂNDIA
2026

SUMÁRIO

1 INTRODUÇÃO

1.1 O problema

1.2 Objetivo e escopo

2 MODELAGEM CONCEITUAL

2.1 Entidades do domínio

2.2 Diagrama de classes

2.3 A distinção entre obra e exemplar

2.4 Organização em pacotes

3 APLICAÇÃO DOS CONCEITOS DE POO

3.1 Classes abstratas

3.2 Herança

3.3 Interfaces

3.4 Polimorfismo

3.5 Encapsulamento

3.6 Composição e agregação

3.7 Membros de classe

3.8 Pacotes e visibilidade

3.9 Sobrecarga

3.10 Tratamento de exceções

4 USO DE BIBLIOTECAS

5 MANUAL DO USUÁRIO

5.1 Requisitos e execução

5.2 Acesso ao sistema

5.3 Empréstimos e devoluções

5.4 Acervo

5.5 Leitores

5.6 Resumo

5.7 Visão do leitor

5.8 Onde os dados ficam gravados

6 VERIFICAÇÃO E TESTES

7 CONCLUSÃO

REFERÊNCIAS

1 INTRODUÇÃO

1.1 O problema

Uma biblioteca acadêmica de porte médio movimenta diariamente dezenas de empréstimos sobre um acervo com milhares de itens de naturezas diferentes. Livros, revistas e mídias em DVD não circulam da mesma forma: um livro técnico costuma ficar semanas com o leitor, enquanto uma revista precisa voltar rápido para atender à fila do mês e um DVD, por existir em poucas cópias, tem prazo ainda menor. Cada um desses tipos cobra multa diferente por dia de atraso.

Além da variedade do acervo, há a variedade de pessoas. O bibliotecário cataloga itens, cadastra leitores e registra empréstimos; o leitor apenas consulta o que está com ele e quando precisa devolver. São papéis com permissões distintas sobre o mesmo conjunto de dados.

Um controle feito em planilha ou em papel falha justamente nos pontos em que o erro custa caro: permite emprestar um item que já saiu, não impede que um leitor acumule mais itens do que o permitido, não calcula a multa correta para cada tipo e não guarda histórico confiável. Esse é o problema que o sistema aqui descrito resolve.

1.2 Objetivo e escopo

O objetivo foi desenvolver, em Java, um sistema orientado a objetos para o controle de acervo, leitores e empréstimos de uma biblioteca, com interface gráfica construída com a biblioteca *Swing* e com os dados gravados em arquivo, de modo que o estado do sistema sobreviva ao encerramento do programa.

O sistema implementado contempla:

- catálogo com três tipos de item, cada um com prazo e multa próprios, admitindo ainda ajuste individual por obra;
- controle de exemplares, já que a biblioteca possui várias cópias do mesmo título;
- cadastro, edição, busca e remoção de obras e de leitores;
- registro de empréstimos e devoluções, com cálculo automático de multa por atraso;
- autenticação com dois perfis, cada um com uma regra e um conjunto de telas;
- persistência em arquivos de texto no formato CSV;
- tratamento de erros por meio de uma hierarquia própria de exceções.

Ficaram fora do escopo recursos que não acrescentariam conceitos novos de orientação a objetos ao trabalho, como reserva de itens com fila de espera, renovação de empréstimo, relatórios gerenciais com gráficos e integração com banco de dados relacional.

2 MODELAGEM CONCEITUAL

2.1 Entidades do domínio

A modelagem partiu da observação do funcionamento real de um balcão de biblioteca. Foram identificadas as entidades apresentadas no Quadro 1, com os papéis que cada uma desempenha.

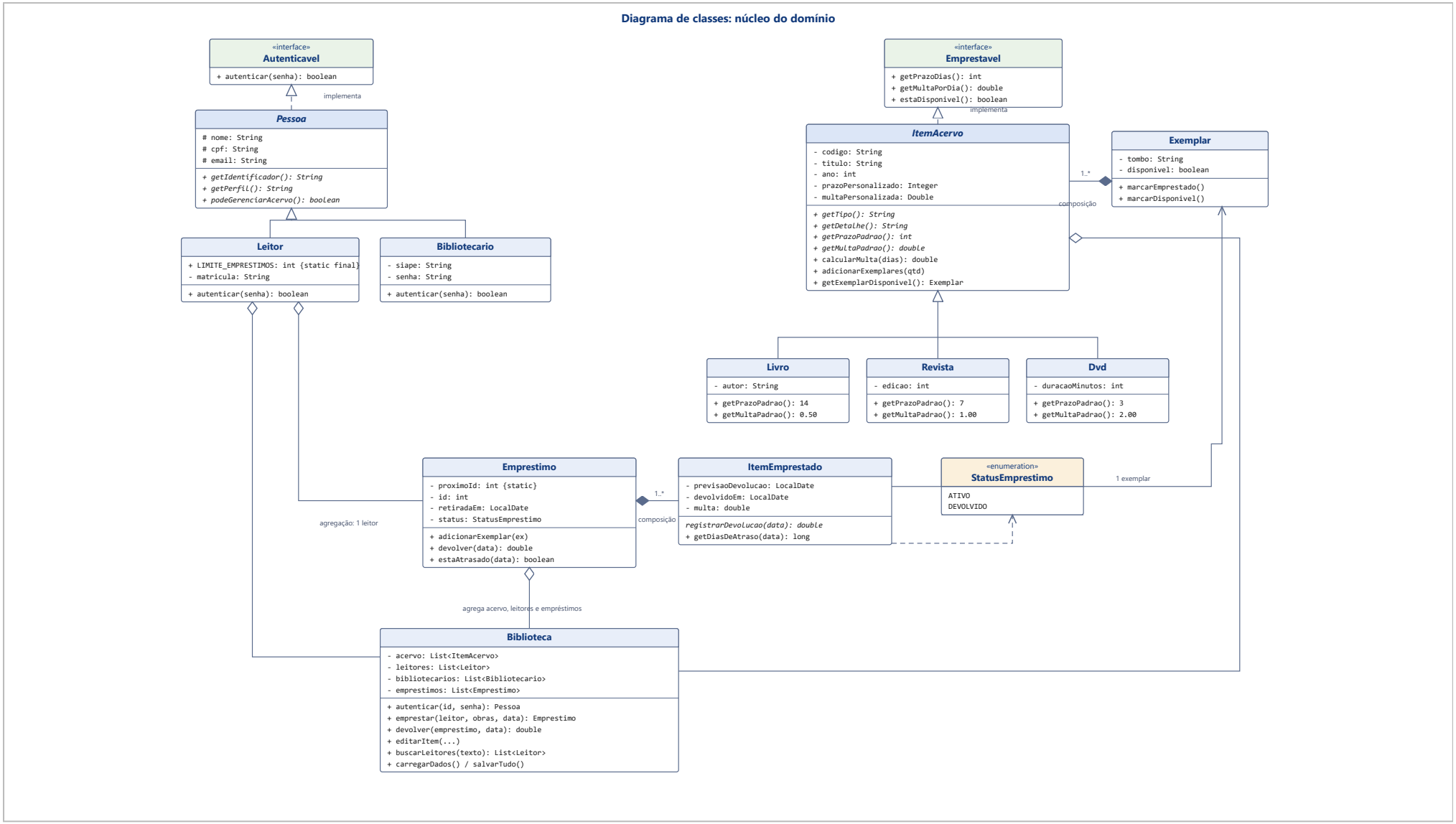
Quadro 1: entidades principais do sistema

Entidade	Papel no sistema
`Pessoa`	Generalização de quem acessa o sistema. Guarda nome, CPF e e-mail, e deixa para as subclasses a regra de autenticação e as permissões.
`Leitor`	Quem retira itens. Possui matrícula e está sujeito ao limite de três itens simultâneos.
`Bibliotecario`	Quem administra o sistema. Possui SIAPE e senha, e é o único que cataloga, edita e registra movimentações.
`ItemAcervo`	A obra catalogada, com código, título e ano. Define prazo e multa, e reúne os seus exemplares.
`Livro, Revista, Dvd`	Especializações da obra. Cada uma guarda o dado que só ela tem (autor, edição, duração) e pratica prazo e multa próprios.
`Exemplar`	A cópia física, identificada por um número de tombo. É o exemplar que sai no empréstimo, não a obra.
`Emprestimo`	A transação: um leitor, uma data de retirada e os itens levados.
`ItemEmprestado`	Cada linha do empréstimo, com a previsão de devolução, a data em que voltou e a multa apurada.
`Biblioteca`	Reúne as listas e concentra as regras de negócio. Nenhuma validação acontece fora dela.

2.2 Diagrama de classes

A Figura 1 apresenta o diagrama de classes do núcleo do domínio. As classes de interface gráfica e de persistência foram omitidas nele para manter a leitura do modelo, e aparecem na Figura 2, que mostra a divisão em pacotes.

Figura 1: diagrama de classes do núcleo do domínio



Fonte: Elaborado pelos autores (2026).

Três relacionamentos merecem destaque no diagrama. A composição entre `ItemAcervo` e `Exemplar` indica que a cópia só existe dentro da obra: remover a obra do acervo elimina os seus exemplares. A mesma relação liga `Emprestimo` e `ItemEmprestado`. Já a ligação entre `Emprestimo` e `Leitor` é de agregação, porque o leitor continua cadastrado depois que o empréstimo é encerrado, e a ligação entre `ItemEmprestado` e `Exemplar` também é, pois o exemplar volta para a estante e segue pertencendo ao acervo.

2.3 A distinção entre obra e exemplar

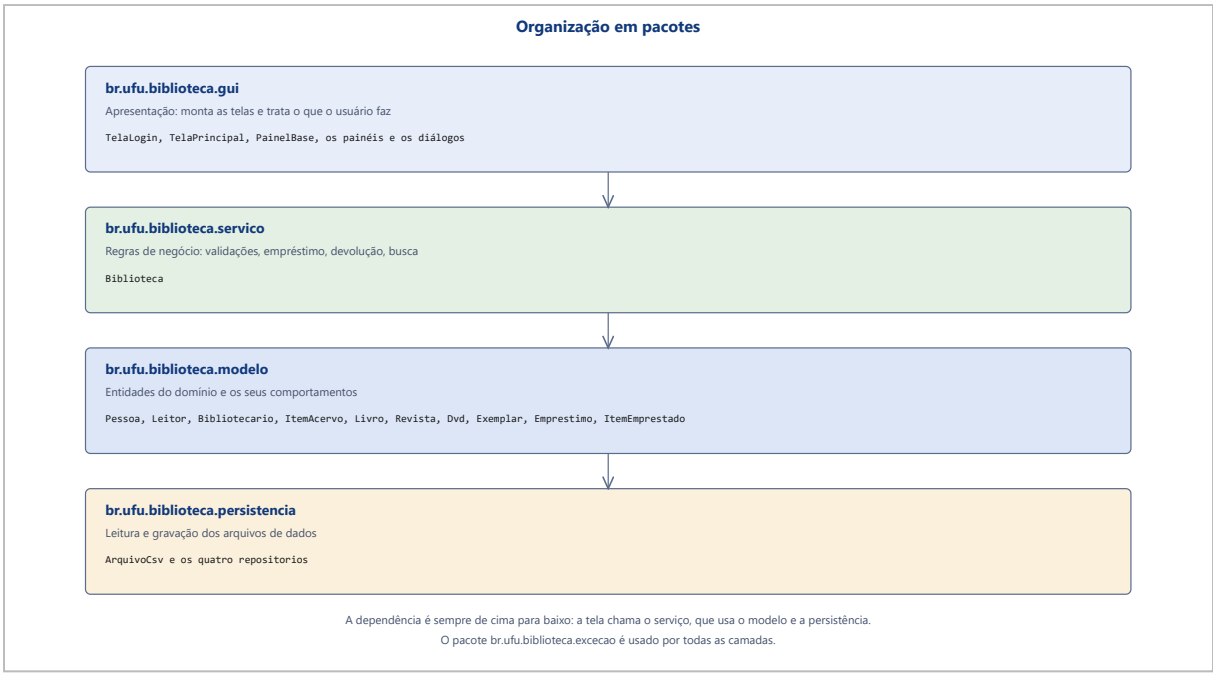
A primeira versão do modelo tratava cada item do acervo como uma peça única. A consequência apareceu assim que o sistema foi exercitado: uma biblioteca que possui quatro cópias do mesmo livro precisaria cadastrar quatro registros com títulos repetidos, e não teria como responder quantas delas estão disponíveis.

O modelo foi então revisto para separar dois conceitos que o senso comum confunde. A obra é o que o catálogo descreve, com título, autor e ano. O exemplar é a cópia física que o leitor leva para casa, identificada por um número de tombo no formato `L001-1`, `L001-2` e assim por diante. Quem é emprestado é o exemplar; quem define prazo e multa é a obra. O leitor escolhe o título e o sistema separa qualquer cópia livre, que é exatamente o que acontece no balcão.

2.4 Organização em pacotes

O código foi dividido em cinco pacotes, com dependência em sentido único, conforme a Figura 2. A interface gráfica conhece o serviço, o serviço conhece o modelo e a persistência, e o modelo não conhece ninguém acima dele. Essa separação permite que as regras sejam testadas sem abrir uma única janela, o que foi feito durante o desenvolvimento, conforme descrito na seção 6.

Figura 2: organização do código em pacotes



Fonte: Elaborado pelos autores (2026).

3 APLICAÇÃO DOS CONCEITOS DE POO

Esta seção percorre os conceitos trabalhados na disciplina mostrando onde cada um aparece no projeto e, principalmente, por que ele foi usado ali. Todos os trechos de código foram extraídos do próprio sistema, sem adaptação.

3.1 Classes abstratas

Duas classes do modelo são abstratas, e pelo mesmo motivo: o conceito que elas representam não existe sozinho no mundo real. Não há como cadastrar uma "pessoa" na biblioteca sem dizer se ela é leitor ou bibliotecário, nem como catalogar um "item" sem dizer se é livro, revista ou DVD. A classe abstrata reúne o que é comum e obriga as subclasses a completarem o que falta.

```
public abstract class Pessoa implements Autenticavel {

    protected String nome;
    protected String cpf;
    protected String email;

    public Pessoa(String nome, String cpf, String email) {
        this.nome = nome;
        this.cpf = cpf;
        this.email = email;
    }

    // Cada subclasse responde o próprio nome de perfil, usado no rodapé da tela.
    public abstract String getPerfil();

    /**
     * Código que a pessoa digita para entrar: matrícula para o leitor, SIAPE
     * para o bibliotecário. Como o método existe aqui, a tela de login percorre
     * uma lista de Pessoa sem precisar saber de qual classe é cada objeto.
     */
    public abstract String getIdentificador();

    // Define o que a tela principal libera para o usuário logado, sem precisar
    // de instanceof espalhado pela interface gráfica.
    public abstract boolean podeGerenciarAcervo();
```

Classe Pessoa: atributos comuns e três métodos abstratos.

O método `getIdentificador()` resolve um problema concreto da tela de login. Leitor entra com matrícula e bibliotecário com SIAPE, mas o método declarado na superclasse permite varrer uma lista de `Pessoa` sem perguntar de qual classe é cada objeto, como mostra a seção 3.4.

3.2 Herança

A hierarquia de pessoas tem dois níveis e duas folhas. O que distingue `Leitor` de `Bibliotecario` não são apenas os atributos, mas a regra de acesso e as permissões, que são comportamento.

```
/**
 * Regra do leitor: entra com os quatro últimos dígitos do CPF. É uma regra
 * mais fraca que a do bibliotecário porque o leitor só consulta dados dele.
 */
@Override
public boolean autenticar(String senha) {
    if (cpf == null || cpf.length() < 4 || senha == null) {
        return false;
    }
    return cpf.substring(cpf.length() - 4).equals(senha);
}
```

Leitor: autenticação pelos quatro últimos dígitos do CPF.

```
/**
 * Regra do bibliotecário: senha cadastrada e com no mínimo seis caracteres.
 * Mesmo nome de método do leitor, comportamento diferente: é a sobrescrita
 * que o polimorfismo usa na tela de login.
 */
@Override
public boolean autenticar(String senha) {
    return this.senha != null && this.senha.equals(senha) && senha.length() >= 6;
}
```

Bibliotecario: mesma assinatura, regra mais rígida.

A hierarquia do acervo segue a mesma ideia. `Livro`, `Revista` e `Dvd` herdam de `ItemAcervo` tudo o que é comum, como código, título, ano e a lista de exemplares, e cada uma informa apenas os valores que pratica:

```
public static final int PRAZO_PADRAO = 3;
public static final double MULTA_PADRAO = 2.00;

// Mídia com poucas cópias: prazo mínimo e multa mais alta.
@Override
public int getPrazoPadrao() {
    return PRAZO_PADRAO;
}
```

Dvd: prazo de três dias, o mais curto do acervo.

3.3 Interfaces

O sistema define duas interfaces. `Autenticavel` declara o contrato de quem entra no sistema, e `Emprestavel` declara o contrato do que pode ser emprestado.

```
public interface Emprestavel {

    int getPrazoDias();

    double getMultaPorDia();

    boolean estaDisponivel();
}
```

Interface Emprestavel: o contrato do que circula pelo empréstimo.

A escolha entre interface e classe abstrata seguiu um critério simples. `ItemAcervo` é uma classe abstrata porque compartilha estado e implementação com as filhas; `Emprestavel` é uma interface porque descreve apenas o que qualquer coisa emprestável precisa responder, sem impor como. Nada impede que, no futuro, um equipamento de laboratório implemente `Emprestavel` sem pertencer ao acervo bibliográfico.

3.4 Polimorfismo

O polimorfismo aparece em quatro pontos do sistema. O primeiro é a autenticação. O método abaixo percorre leitores e bibliotecários na mesma lista e chama `autenticar()` sem saber com qual perfil está lidando: quem decide a regra é o objeto, em tempo de execução.

```
/**
 * Procura a pessoa pelo identificador e deixa que ela mesma valide a senha.
 * Bibliotecário e leitor usam regras diferentes, mas a chamada é a mesma:
 * é a sobrescrita de autenticar() que decide o resultado.
 */
public Pessoa autenticar(String identificador, String senha) throws AutenticacaoException {
    List<Pessoa> usuarios = new ArrayList<>();
    usuarios.addAll(bibliotecarios);
    usuarios.addAll(leitores);

    for (Pessoa pessoa : usuarios) {
        if (pessoa.getIdentificador().equalsIgnoreCase(identificador)
                && pessoa.autenticar(senha)) {
            return pessoa;
        }
    }

    throw new AutenticacaoException();
}
```

Autenticação polimórfica: uma chamada, duas regras possíveis.

O segundo ponto é o cálculo da multa. O método `calcularMulta()` é concreto e está escrito uma única vez na superclasse, mas o valor diário vem de `getMultaPorDia()`, que por sua vez consulta o padrão da subclasse. O mesmo código produz R$ 0,50 por dia para um livro e R$ 2,00 para um DVD.

```
/**
 * A conta é a mesma para toda obra, o que muda é o valor diário. Por isso
 * o método é concreto aqui e chama getMultaPorDia(), que cai no padrão da
 * subclasse ou no ajuste feito para esta obra.
 */
public double calcularMulta(long diasDeAtraso) {
    if (diasDeAtraso <= 0) {
        return 0.0;
    }
    return diasDeAtraso * getMultaPorDia();
}
```

Cálculo único da multa, com valor diário resolvido em tempo de execução.

O terceiro ponto está na gravação do acervo. O repositório grava qualquer item chamando `getTipo()` e `getDetalhe()`, sem testar a classe de cada objeto. Acrescentar um quarto tipo de item ao sistema não exigiria alterar uma linha do repositório.

O quarto ponto está na própria interface gráfica. Todas as abas herdam de `PainelBase`, que declara `atualizar()` como abstrato. Quando o usuário troca de aba, a janela principal chama esse método sem saber qual painel está aberto.

```
private void atualizarAbaSelecionada() {
    java.awt.Component selecionada = abas.getSelectedComponent();
    // Todas as abas herdam de PainelBase, então a chamada é a mesma para
    // qualquer uma delas: entra em ação a versão de atualizar() do painel.
    if (selecionada instanceof PainelBase) {
        ((PainelBase) selecionada).atualizar();
    }
}
```

Troca de aba: a janela não conhece o painel, apenas o contrato.

3.5 Encapsulamento

Todos os atributos do modelo são privados, com acesso controlado por métodos. Em dois casos o encapsulamento vai além do atributo e protege a própria estrutura. O empréstimo devolve a sua lista de itens em modo somente leitura, impedindo que outra classe insira ou remova linhas por fora dele:

```
/**
 * Devolve a lista protegida contra alteração. Quem recebe consegue ler os
 * itens, mas não consegue inserir nem remover por fora do empréstimo, que
 * é o dono deles.
 */
public List<ItemEmprestado> getItens() {
    return Collections.unmodifiableList(itens);
}
```

A lista é exposta sem permitir alteração externa.

O segundo caso é o ajuste de prazo e multa. Em vez de um `set` direto, o método só guarda o valor quando ele difere do padrão do tipo, mantendo a obra ligada à regra da sua classe sempre que possível:

```
/**
 * Guarda o ajuste apenas quando ele difere do padrão do tipo. Assim uma
 * obra que ficou com o valor de fábrica continua acompanhando a regra da
 * classe mesmo que o padrão mude depois.
 */
public void definirPrazo(int dias) {
    this.prazoPersonalizado = (dias == getPrazoPadrao()) ? null : dias;
}
```

O objeto decide como guardar o valor que recebe.

3.6 Composição e agregação

A diferença entre as duas relações foi tratada com rigor no projeto, inclusive na visibilidade dos construtores. Na composição, o objeto contido não pode existir sem o seu dono, e no código isso é garantido pelo construtor com visibilidade de pacote: só `ItemAcervo` cria exemplares, e só `Emprestimo` cria itens emprestados.

```
public class Exemplar {

    private final ItemAcervo obra;
    private final String tombo;
    private boolean disponivel;

    Exemplar(ItemAcervo obra, String tombo) {
        this.obra = obra;
        this.tombo = tombo;
        this.disponivel = true;
    }
```

Exemplar: construtor sem modificador, acessível apenas dentro do pacote.

Na agregação, os objetos existem de forma independente. O empréstimo referencia um leitor que continua cadastrado depois da devolução, e cada item emprestado referencia um exemplar que volta para a estante. A classe `Biblioteca` também agrega: ela mantém as listas, mas não é dona da existência de nenhum dos objetos.

3.7 Membros de classe

O sistema usa membros estáticos em três situações, sempre quando a informação pertence ao conjunto e não a um objeto específico. O limite de empréstimos vale para a biblioteca inteira, e por isso é uma constante de classe:

```
// Constante de classe: o limite vale para a biblioteca inteira, não para
// um leitor específico, por isso é static final.
public static final int LIMITE_EMPRESTIMOS = 3;
```

Constante usada pelas regras e também exibida nas telas.

O número do empréstimo é gerado por um contador estático, que precisa ser sincronizado com o que já foi gravado em arquivo. A solução ficou no segundo construtor, mostrado na seção 3.9.

Por fim, `ArquivoCsv` é uma classe utilitária: todos os seus métodos são estáticos e o construtor é privado, justamente para impedir que alguém a instancie sem necessidade.

3.8 Pacotes e visibilidade

Os cinco pacotes do sistema seguem o padrão de domínio invertido e isolam responsabilidades, como mostrado na Figura 2. A visibilidade dos membros acompanha essa divisão: atributos privados no modelo, `protected` apenas nos campos de `Pessoa` que as subclasses realmente usam, visibilidade de pacote nos construtores das classes compostas e `public` somente no que outras camadas precisam chamar.

3.9 Sobrecarga

A classe `Emprestimo` tem dois construtores. O primeiro é usado quando um empréstimo nasce na tela e precisa de um número novo; o segundo, quando ele é remontado a partir do arquivo e precisa manter o número que já tinha. O primeiro delega ao segundo com `this(...)`, de modo que a inicialização acontece em um lugar só.

```
// Empréstimo novo: o id sai do contador.
public Emprestimo(Leitor leitor, LocalDate retiradaEm) {
    this(proximoId++, leitor, retiradaEm, StatusEmprestimo.ATIVO);
}
```

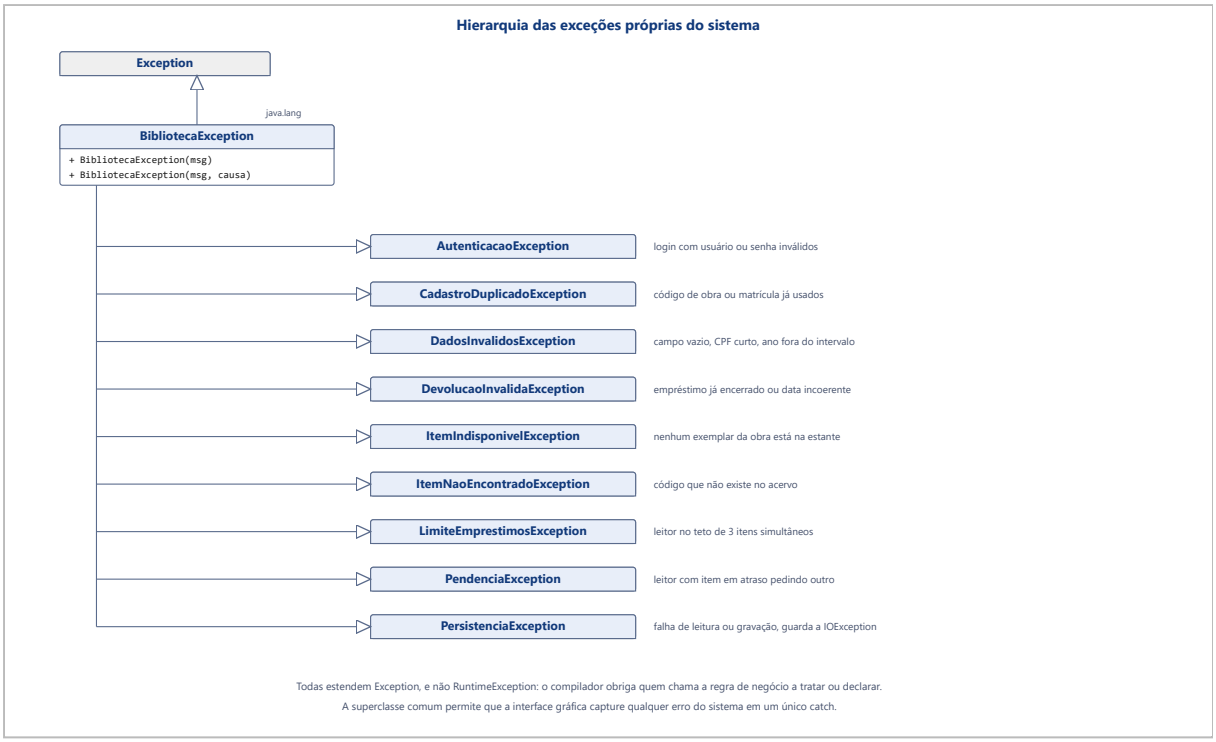
Dois construtores com listas de parâmetros diferentes.

O mesmo recurso aparece em `DialogoItem`, cuja janela serve para cadastrar e para editar. O construtor com dois parâmetros chama o de três passando `null`, e é esse valor que diferencia os dois modos de operação da tela.

3.10 Tratamento de exceções

O sistema define dez exceções próprias, organizadas na hierarquia da Figura 3. Todas descendem de `BibliotecaException`, que por sua vez estende `Exception` e não `RuntimeException`. A escolha é deliberada: como são erros previsíveis do negócio, o compilador deve obrigar quem chama a regra a decidir o que fazer com eles.

Figura 3: hierarquia das exceções próprias do sistema



Fonte: Elaborado pelos autores (2026).

O método de empréstimo concentra as regras e, com elas, as exceções. As verificações são feitas na ordem da mais barata para a mais cara, e o objeto só é criado depois que todas passam, de modo que um empréstimo nunca fica pela metade:

```
/**
 * Regras aplicadas em ordem, da mais barata para a mais cara de verificar.
 * Cada uma lança a sua própria exceção, então a tela sabe exatamente o que
 * mostrar ao usuário sem precisar interpretar códigos de retorno.
 */
public Emprestimo emprestar(Leitor leitor, List<ItemAcervo> obrasEscolhidas, LocalDate
retiradaEm)
        throws DadosInvalidosException, ItemIndisponivelException,
        LimiteEmprestimosException, PendenciaException, PersistenciaException {

    if (leitor == null) {
        throw new DadosInvalidosException("Escolha o leitor.");
    }
    if (obrasEscolhidas == null || obrasEscolhidas.isEmpty()) {
        throw new DadosInvalidosException("Marque pelo menos um item do catálogo.");
    }
    if (retiradaEm.isAfter(LocalDate.now())) {
        throw new DadosInvalidosException("A data de retirada não pode estar no futuro.");
    }
    // Sem um limite para trás, um ano digitado errado geraria anos de
    // atraso e uma multa impossível.
    LocalDate maisAntiga = LocalDate.now().minusYears(1);
    if (retiradaEm.isBefore(maisAntiga)) {
        throw new DadosInvalidosException(
                "A data de retirada não pode ser anterior a " + texto(maisAntiga) + ".");
    }
    if (temAtraso(leitor, LocalDate.now())) {
        throw new PendenciaException(leitor.getNome());
    }

    int emMaos = contarItensEmMaos(leitor);
    if (emMaos + obrasEscolhidas.size() > Leitor.LIMITE_EMPRESTIMOS) {
        throw new LimiteEmprestimosException(leitor.getNome(), Leitor.LIMITE_EMPRESTIMOS);
    }

    for (ItemAcervo obra : obrasEscolhidas) {
        if (!obra.estaDisponivel()) {
            throw new ItemIndisponivelException(obra.getTitulo());
        }
        // Levar duas cópias do mesmo título não ajuda o leitor e tira da
        // estante um exemplar que outra pessoa poderia estar usando.
        if (jaEstaCom(leitor, obra)) {
            throw new DadosInvalidosException(leitor.getNome()
                    + " já está com um exemplar de \"" + obra.getTitulo() + "\".");
        }
    }

    // Só aqui o objeto é criado: nenhuma regra pode falhar depois deste
    // ponto, então o empréstimo nunca fica pela metade.
    Emprestimo emprestimo = new Emprestimo(leitor, retiradaEm);
    for (ItemAcervo obra : obrasEscolhidas) {
        // O leitor escolhe o título, o sistema escolhe a cópia.
        emprestimo.adicionarExemplar(obra.getExemplarDisponivel());
    }
    emprestimos.add(emprestimo);

    salvarTudo();
    return emprestimo;
}
```

Regras do empréstimo, cada uma com a sua própria exceção.

Na camada de persistência aparecem outras duas técnicas. O `try-with-resources` fecha o arquivo mesmo quando a leitura falha no meio, e a `IOException` original é guardada como causa da exceção do sistema, de modo que o motivo técnico não se perde quando a tela mostra uma mensagem simples ao usuário. Note também que arquivo inexistente não é tratado como erro: na primeira execução, a lista vazia é a resposta correta.

```
public static List<String[]> ler(String arquivo) throws PersistenciaException {
    List<String[]> linhas = new ArrayList<>();
    Path caminho = PASTA.resolve(arquivo);

    // Primeira execução do sistema: ainda não existe arquivo, e isso não é
    // erro. Devolver a lista vazia deixa o programa abrir normalmente.
    if (!Files.exists(caminho)) {
        return linhas;
    }

    // try-with-resources: o BufferedReader é fechado sozinho no fim do
    // bloco, inclusive se a leitura lançar exceção no meio do arquivo.
    try (BufferedReader leitor = Files.newBufferedReader(caminho, StandardCharsets.UTF_8)) {
        String linha;
        while ((linha = leitor.readLine()) != null) {
            if (!linha.trim().isEmpty()) {
                // o -1 mantém os campos vazios do fim da linha
                linhas.add(linha.split(SEPARADOR, -1));
            }
        }
    } catch (IOException e) {
        // A IOException vira uma exceção do sistema, mas segue guardada
        // como causa para não perder o motivo técnico da falha.
        throw new PersistenciaException("Falha ao ler o arquivo " + arquivo + ".", e);
    }

    return linhas;
}
```

Leitura com fechamento automático e encadeamento da causa.

A interface gráfica fecha o ciclo. Como todas as exceções têm uma superclasse comum, um único `catch` trata limite atingido, obra esgotada, leitor em atraso, data inválida e falha de gravação, sempre com a mensagem já pronta para o usuário. O bloco `finally` devolve o cursor ao estado normal independentemente do que aconteça:

```
private void emprestar() {
    setCursor(Cursor.getPredefinedCursor(Cursor.WAIT_CURSOR));
    try {
        // Encerra a edição em andamento, senão o último clique na caixa de
        // marcação ainda não teria chegado ao modelo da tabela.
        if (tabelaCatalogo.isEditing()) {
            tabelaCatalogo.getCellEditor().stopCellEditing();
        }

        List<ItemAcervo> escolhidas = getObrasMarcadas();
        LocalDate retirada = Formatos.lerData(campoRetirada.getText(), "data de retirada");

        Emprestimo emprestimo = biblioteca.emprestar(leitor, escolhidas, retirada);

        JOptionPane.showMessageDialog(this, montarRecibo(emprestimo),
                "Empréstimo registrado", JOptionPane.INFORMATION_MESSAGE);
        atualizar();

    } catch (BibliotecaException e) {
        // Limite atingido, obra sem exemplar livre, leitor em atraso, data
        // inválida ou erro de gravação: todas chegam com mensagem pronta.
        JOptionPane.showMessageDialog(this, e.getMessage(),
                "Empréstimo não realizado", JOptionPane.WARNING_MESSAGE);
    } finally {
        // O finally garante que o cursor volte ao normal mesmo quando a
        // operação termina em exceção.
        setCursor(Cursor.getDefaultCursor());
    }
}
```

Tratamento na tela: captura única, mensagem ao usuário e finally.

Há ainda a conversão de erros técnicos em linguagem de usuário. Quando alguém digita letras em um campo numérico, a `NumberFormatException` do `parse` é capturada e relançada como `DadosInvalidosException` com um texto compreensível, em vez de subir até a tela como erro de programação.

```
private int numeroPositivo(String valor, String campo) throws DadosInvalidosException {
    exigirTexto(valor, campo);
    try {
        int numero = Integer.parseInt(valor.trim());
        if (numero <= 0) {
            throw new DadosInvalidosException("O campo " + campo + " deve ser maior que
zero.");
        }
        return numero;
    } catch (NumberFormatException e) {
        // converte o erro técnico do parse em uma mensagem que o usuário entende
        throw new DadosInvalidosException("O campo " + campo + " aceita apenas números.");
    }
}
```

Erro técnico traduzido para uma mensagem que o usuário entende.

4 USO DE BIBLIOTECAS

O projeto usa apenas bibliotecas da plataforma Java padrão, sem dependências externas, o que permite compilar e executar o sistema em qualquer máquina com o JDK instalado. O Quadro 2 relaciona os pacotes utilizados e a função de cada um.

Quadro 2: bibliotecas utilizadas

Biblioteca	Uso no sistema
`javax.swing`	Toda a interface gráfica. Janelas (`JFrame`), diálogos modais (`JDialog`), abas (`JTabbedPane`), tabelas (`JTable` com `DefaultTableModel`), formulários e as mensagens de aviso (`JOptionPane`).
`java.awt`	Gerenciadores de layout (`BorderLayout`, `GridLayout`, `FlowLayout`), cores, fontes e o cursor de espera.
`java.time`	Datas de retirada, previsão e devolução com `LocalDate`, contagem de dias de atraso com `ChronoUnit` e conversão para o formato brasileiro com `DateTimeFormatter`.
`java.nio.file`	Leitura e gravação dos arquivos CSV em UTF-8, criação da pasta de dados e verificação de existência dos arquivos.
`java.io`	`BufferedReader` e `BufferedWriter` dentro do `try-with-resources`, e a `IOException` encadeada nas exceções do sistema.
`java.util`	Coleções (`List`, `ArrayList`, `Map`, `LinkedHashMap`) e `Collections.unmodifiableList` para proteger as listas expostas.

A escolha do *Swing* atendeu à exigência do enunciado e trouxe uma vantagem prática: por fazer parte do JDK, o sistema não precisa de instalação de bibliotecas para ser demonstrado em qualquer computador.

5 MANUAL DO USUÁRIO

5.1 Requisitos e execução

O sistema exige apenas o JDK 17 ou superior. Há duas formas de executá-lo. A primeira é abrir a pasta `src` em uma IDE, como IntelliJ IDEA, Eclipse ou NetBeans, e executar a classe `br.ufu.biblioteca.Main`. A segunda é dar um duplo clique no arquivo `executar.bat`, que localiza o Java instalado, compila o projeto e abre a janela de acesso.

Pela linha de comando, os dois passos são:

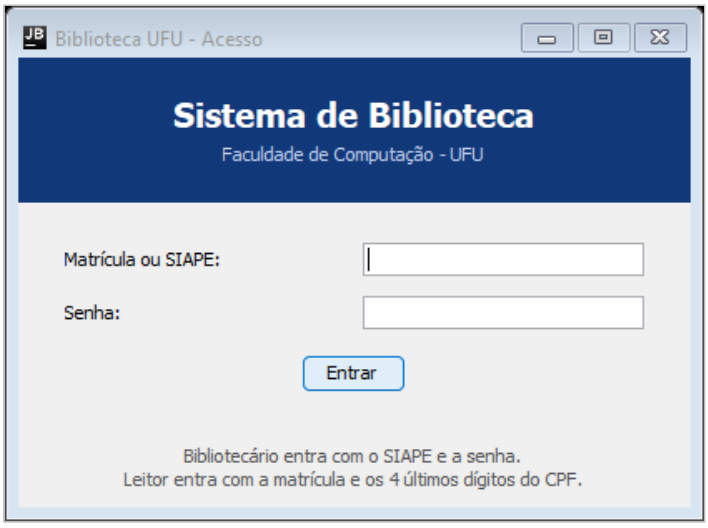
```
javac -encoding UTF-8 -sourcepath src -d build src/br/ufu/biblioteca/Main.java
java -cp build br.ufu.biblioteca.Main
```

Na primeira execução, o sistema percebe que ainda não há dados gravados e cria uma carga inicial com nove obras, vinte e quatro exemplares, oito leitores e um bibliotecário, de modo que já é possível usar todas as funções sem cadastrar nada.

5.2 Acesso ao sistema

A tela de acesso pede um identificador e uma senha. O bibliotecário entra com o número SIAPE e a sua senha; o leitor entra com a matrícula e os quatro últimos dígitos do CPF. A própria tela lembra essas duas regras no rodapé.

Figura 4: tela de acesso ao sistema



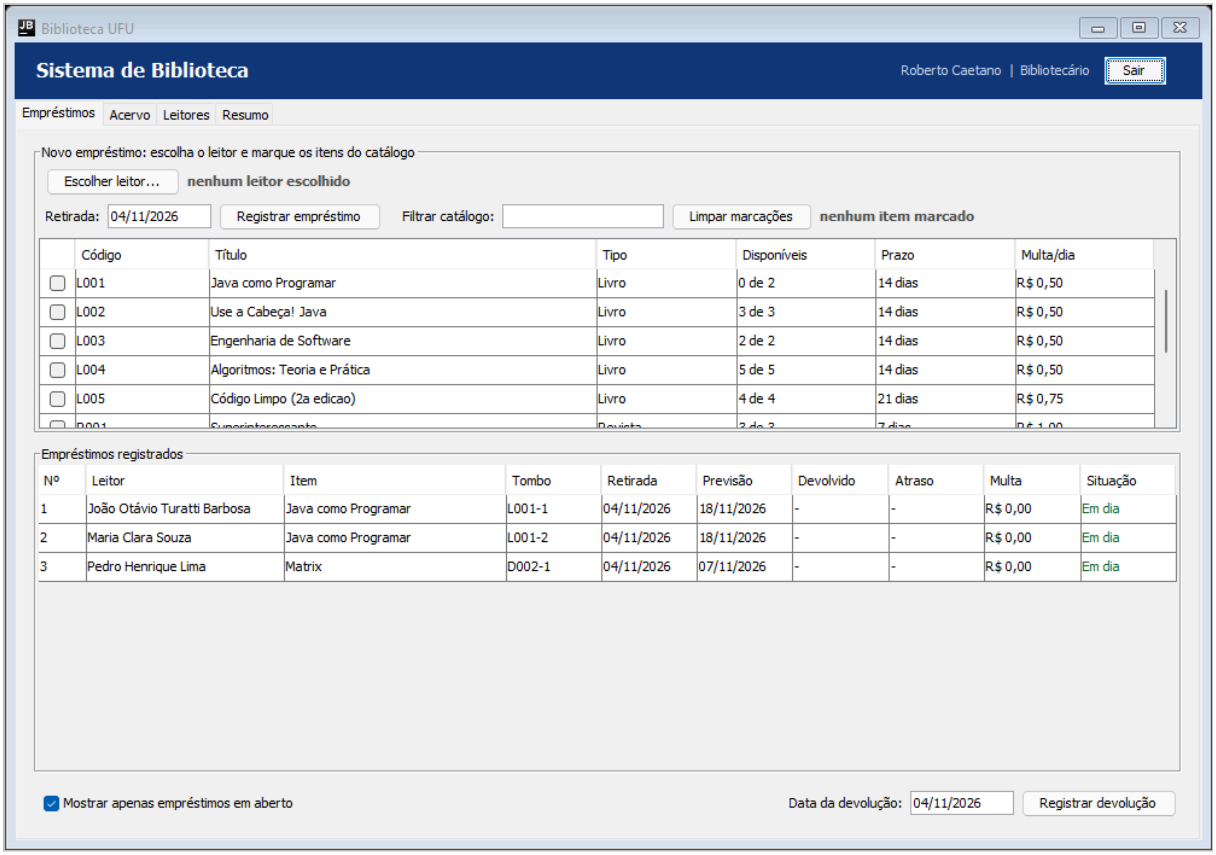
Fonte: Elaborado pelos autores (2026).

O acesso do bibliotecário criado na carga inicial é o SIAPE `1001` com a senha `ufu2026`. Qualquer leitor da lista inicial entra com a sua matrícula e os quatro últimos dígitos do CPF mostrados na aba Leitores.

5.3 Empréstimos e devoluções

Esta é a tela principal do bibliotecário. O leitor é localizado pelo botão *Escolher leitor*, que abre uma busca por matrícula, nome ou CPF. Os itens são marcados na tabela do catálogo, que mostra quantos exemplares de cada obra estão disponíveis, além do prazo e da multa praticados.

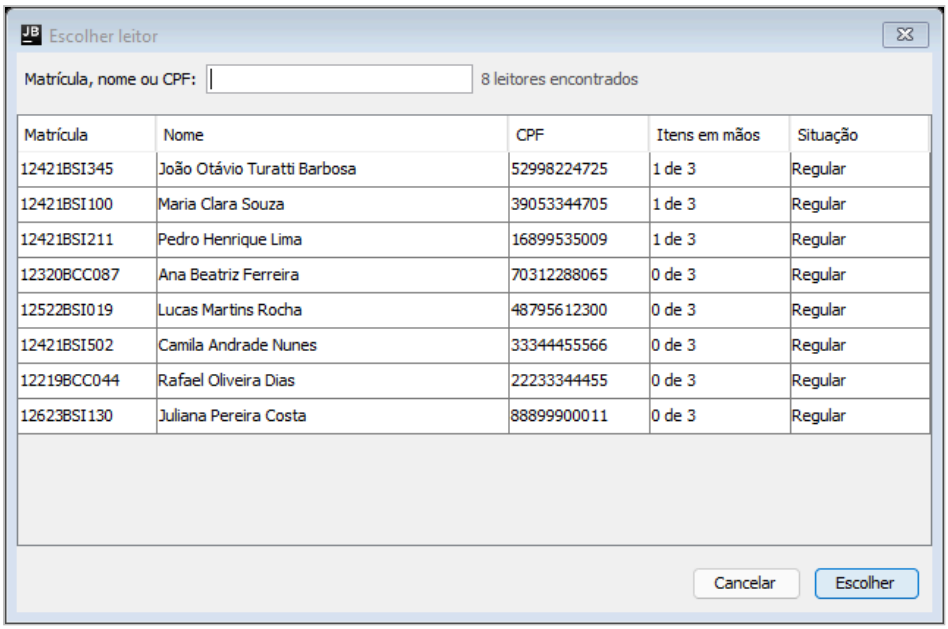
Figura 5: aba de empréstimos e devoluções



Fonte: Elaborado pelos autores (2026).

A busca de leitor, mostrada na Figura 6, exibe também quantos itens cada um está com ele e se há atraso, o que evita iniciar um empréstimo que seria recusado adiante.

Figura 6: busca de leitor



Fonte: Elaborado pelos autores (2026).

Para registrar um empréstimo:

1. clique em *Escolher leitor* e localize o leitor;
2. marque na tabela os títulos que ele vai levar;
3. confirme ou ajuste a data de retirada, que já vem preenchida com a data de hoje;
4. clique em *Registrar empréstimo*.

O sistema responde com um recibo que traz o número do empréstimo, o tombo de cada exemplar separado e a data limite de devolução de cada um, que é diferente conforme o tipo do item. Para devolver, basta selecionar a linha do empréstimo na tabela de baixo, conferir a data e clicar em *Registrar devolução*. Havendo atraso, a multa é calculada e exibida na confirmação.

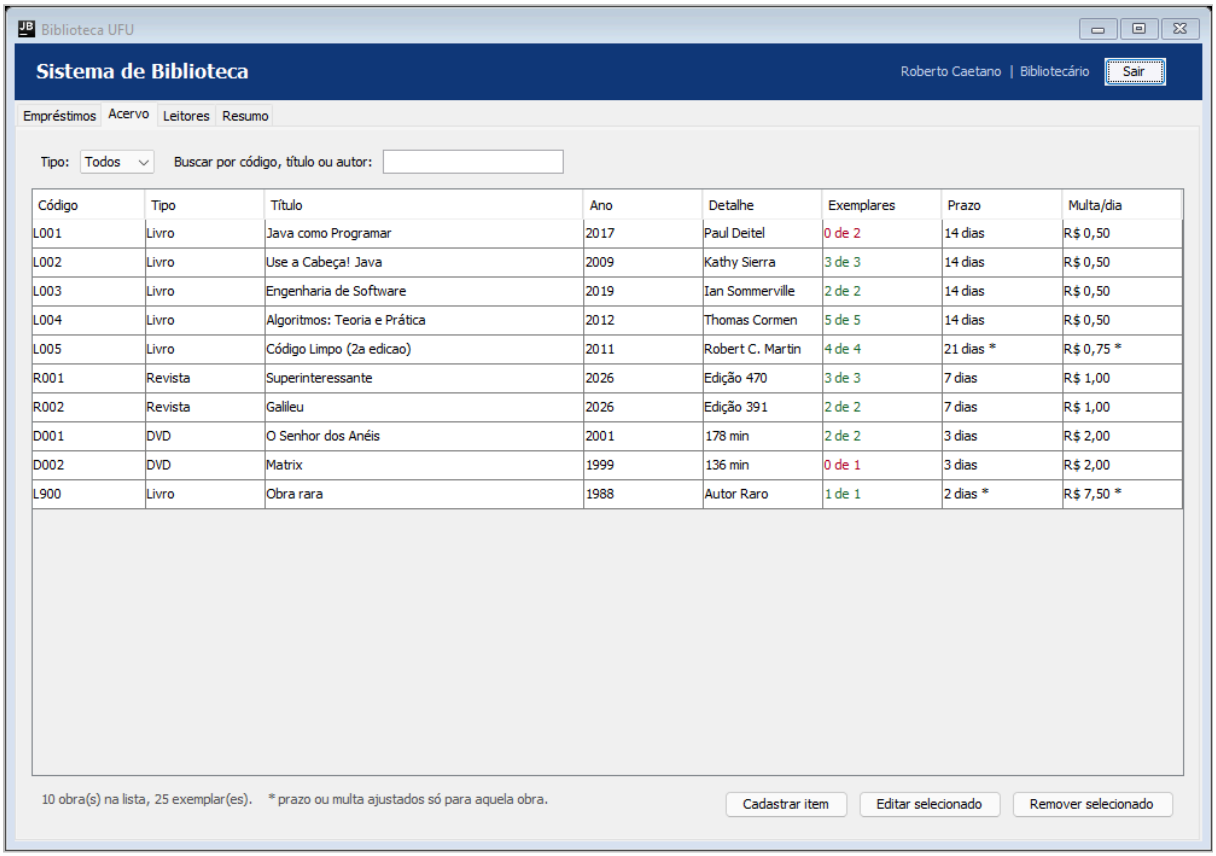
As recusas possíveis, todas com mensagem explicativa, são:

- leitor já está com três itens;
- leitor tem item em atraso;
- nenhum exemplar da obra está disponível;
- leitor já está com um exemplar daquele mesmo título;
- data de retirada no futuro ou anterior a um ano atrás;
- data de devolução no futuro ou anterior à retirada.

5.4 Acervo

A aba Acervo lista as obras catalogadas, com filtro por tipo e busca por código, título ou autor. A coluna de exemplares mostra quantos estão na estante em relação ao total, em vermelho quando a obra está esgotada. O asterisco ao lado do prazo ou da multa indica que aquela obra tem valores próprios, diferentes do padrão do seu tipo.

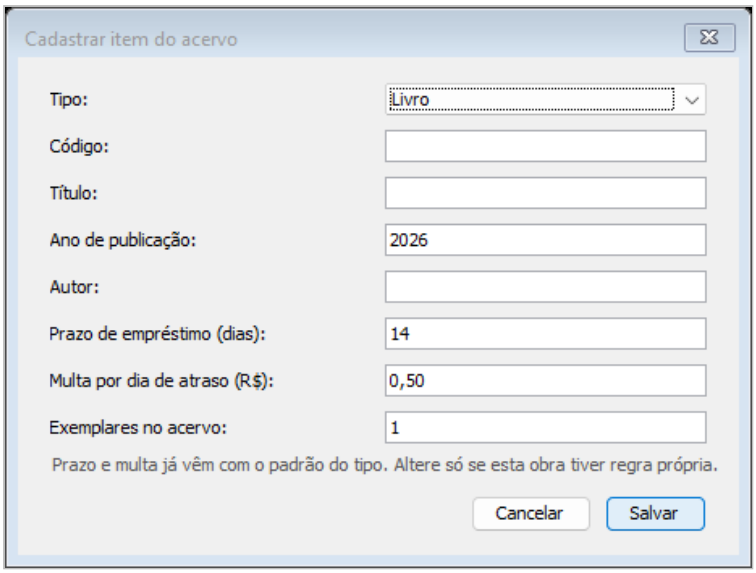
Figura 7: aba do acervo



Fonte: Elaborado pelos autores (2026).

O botão *Cadastrar item* abre o formulário da Figura 8. O campo de detalhe muda de rótulo conforme o tipo escolhido, pedindo autor para livro, edição para revista e duração para DVD. Prazo e multa já vêm preenchidos com o padrão do tipo e só precisam ser alterados quando a obra tiver regra própria.

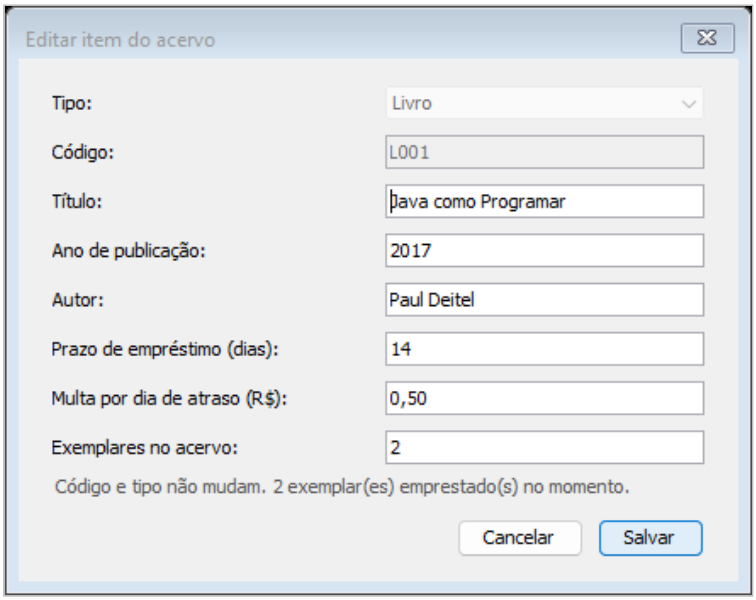
Figura 8: cadastro de item do acervo



Fonte: Elaborado pelos autores (2026).

Para alterar uma obra já cadastrada, selecione a linha e clique em *Editar selecionado*, ou simplesmente dê um duplo clique nela. Na edição, o código e o tipo ficam bloqueados, porque identificam o registro e os tombos já atribuídos às cópias. A quantidade de exemplares pode ser aumentada ou reduzida, respeitando o que está emprestado no momento.

Figura 9: edição de item, com código e tipo bloqueados

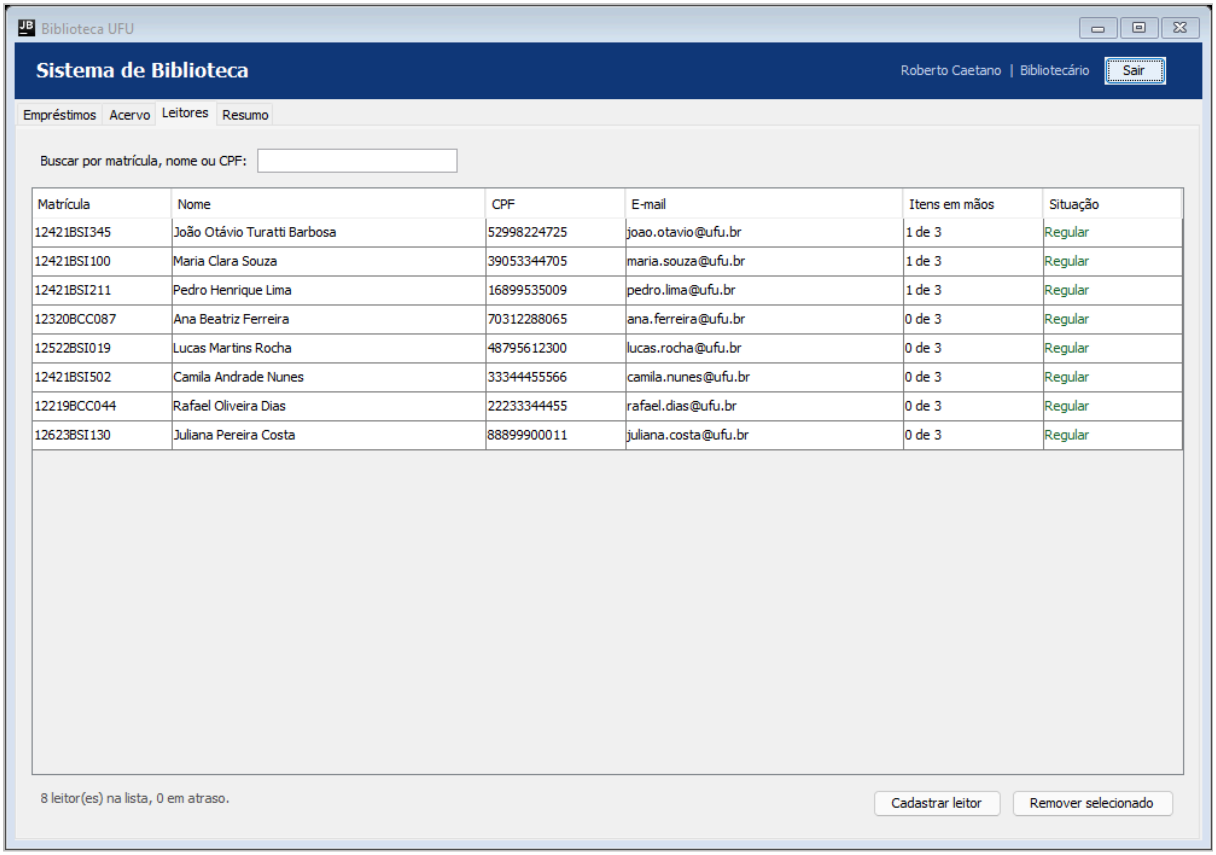


Fonte: Elaborado pelos autores (2026).

5.5 Leitores

A aba Leitores funciona de forma parecida, com busca por matrícula, nome ou CPF, e mostra quantos itens cada leitor está levando e se está regular ou em atraso. O cadastro pede matrícula, nome, CPF e e-mail, e lembra que a senha do leitor será formada pelos quatro últimos dígitos do CPF informado.

Figura 10: aba de leitores

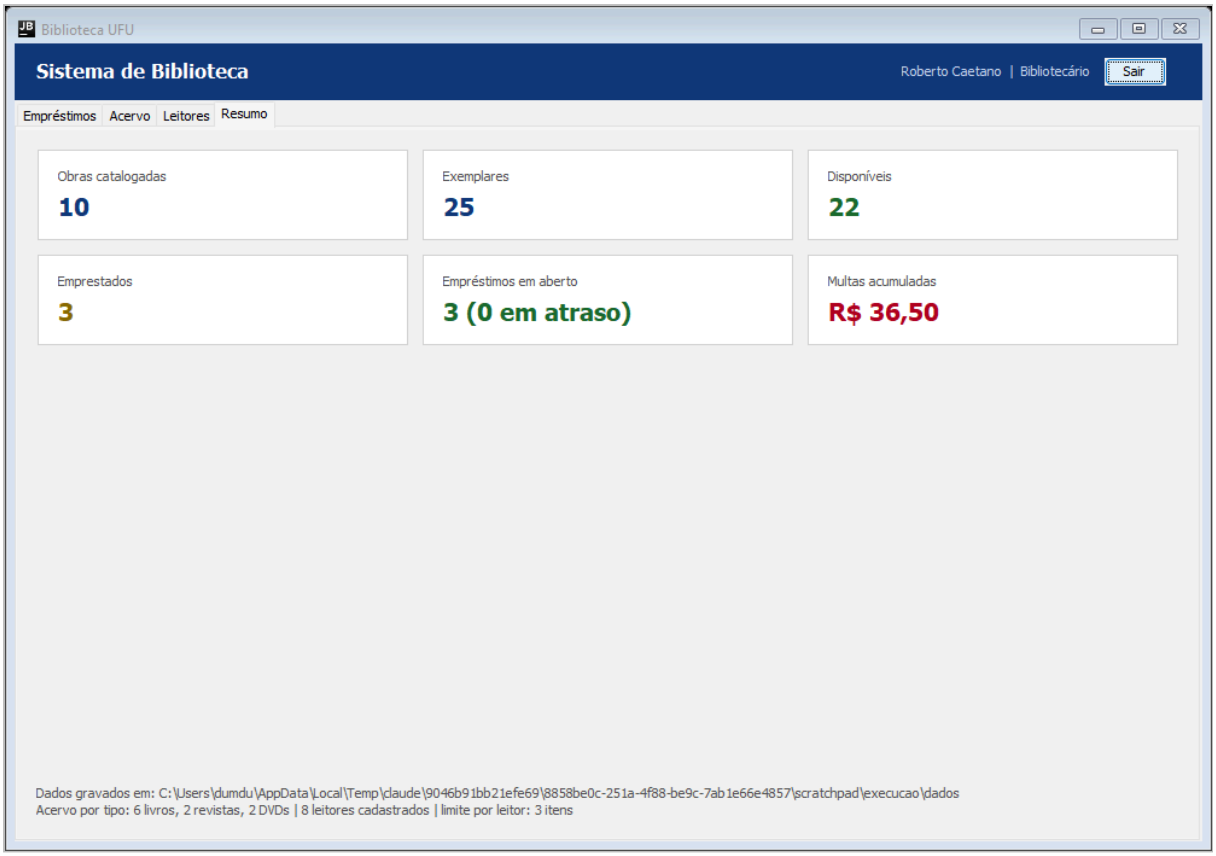


Fonte: Elaborado pelos autores (2026).

5.6 Resumo

A aba Resumo apresenta os números gerais da biblioteca e informa em qual pasta os dados estão sendo gravados, o que é útil para conferir os arquivos durante uma demonstração.

Figura 11: aba de resumo

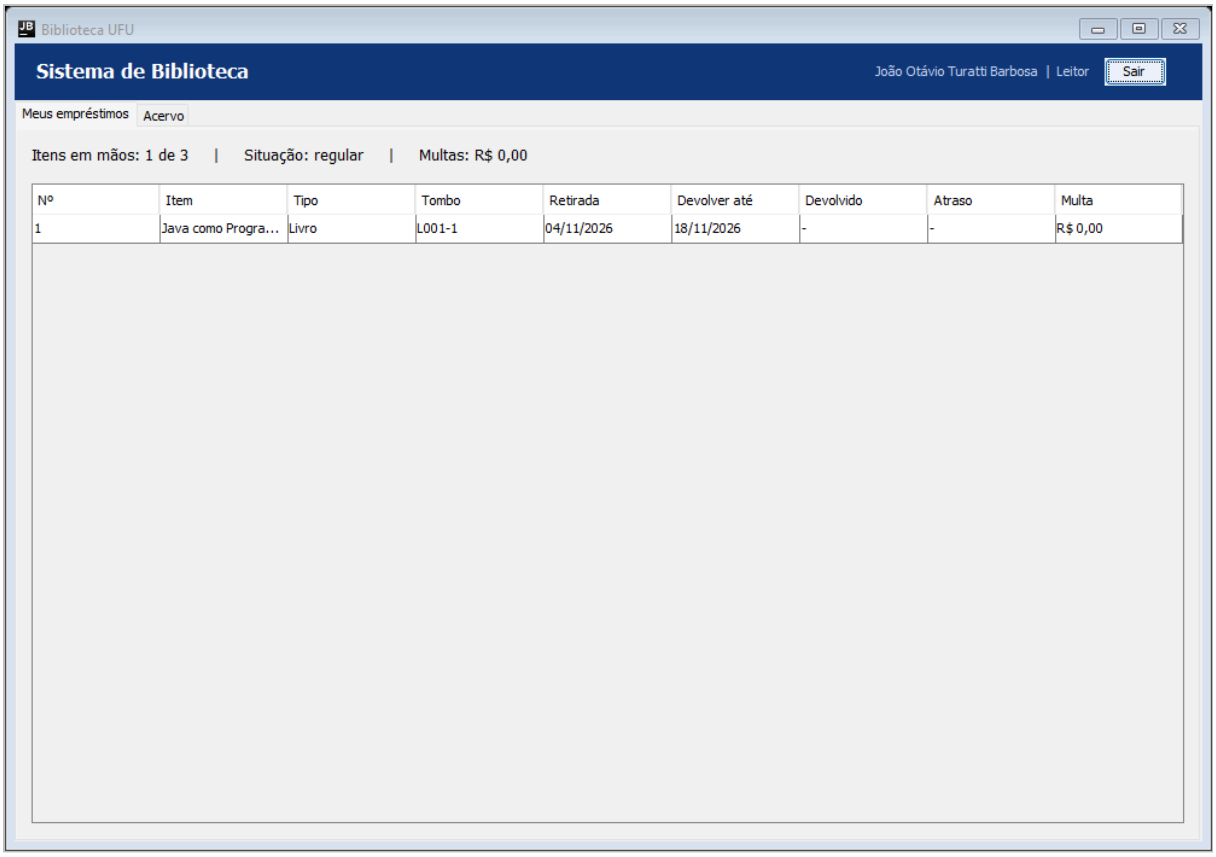


Fonte: Elaborado pelos autores (2026).

5.7 Visão do leitor

Quando quem entra é um leitor, a janela monta um conjunto diferente de abas. Ele vê os próprios empréstimos, com o tombo do exemplar, a data limite e a multa, além de poder consultar o acervo. Nenhum botão de cadastro, edição ou remoção é criado, porque a decisão parte do próprio objeto, pelo método `podeGerenciarAcervo()`.

Figura 12: visão do leitor



Fonte: Elaborado pelos autores (2026).

O botão *Sair*, no canto superior direito, encerra a sessão e devolve a tela de acesso sem fechar o programa, permitindo alternar entre os dois perfis.

5.8 Onde os dados ficam gravados

Os dados são gravados em cinco arquivos de texto, na pasta `dados`, dentro do diretório de execução. O formato CSV foi escolhido por ser legível em qualquer editor de texto, o que permite inspecionar o conteúdo sem ferramentas adicionais.

Quadro 3: arquivos de dados

Arquivo	Conteúdo de cada linha
`acervo.csv`	`tipo;codigo;titulo;ano;detalhe;prazo;multa`
`exemplares.csv`	`codigoDaObra;tombo;disponivel`
`leitores.csv`	`matricula;nome;cpf;email`
`bibliotecarios.csv`	`siape;nome;cpf;email;senha`
`emprestimos.csv`	`id;matricula;retirada;status;tombo;previsao;devolucao;multa`

As colunas de prazo e multa do acervo ficam vazias quando a obra segue o padrão do seu tipo, e os empréstimos repetem o cabeçalho em cada linha de item, sendo agrupados pelo número na leitura. Apagar a pasta `dados` faz o sistema recriar a carga inicial na próxima abertura.

5.9 Repositório

O projeto completo está disponível em: *[endereço do repositório no GitHub]*

6 VERIFICAÇÃO E TESTES

Como as regras de negócio ficaram isoladas na classe `Biblioteca`, foi possível exercitá-las sem abrir a interface gráfica. Durante o desenvolvimento foi mantida uma classe de teste que executa oitenta e uma verificações a cada alteração do código, cobrindo os casos de sucesso e, principalmente, os de erro.

Entre as situações verificadas estão:

- empréstimo do mesmo título para leitores diferentes, conferindo que saem tombos distintos;
- obra com um único exemplar esgotando após o primeiro empréstimo;
- recusa das nove exceções de negócio, uma a uma;
- cálculo da multa para os três tipos de item e para uma obra com valor próprio;
- edição de obra, inclusive a redução de exemplares abaixo do que está emprestado;
- gravação e releitura completa dos arquivos, conferindo tombos, ajustes e multas.

O último grupo de verificações merece destaque: após cada bateria, uma segunda instância da biblioteca é carregada do zero a partir dos arquivos e comparada com a primeira. É o que garante que a persistência não perde nem distorce informação.

A interface gráfica foi verificada por inspeção visual de cada tela e pela execução dos fluxos descritos no manual.

7 CONCLUSÃO

O sistema desenvolvido cumpre o que o trabalho pedia: modela um problema real com orientação a objetos, aplica os conceitos da disciplina onde eles resolvem algo concreto, guarda os dados em arquivo e oferece uma interface gráfica com a qual o usuário opera a biblioteca do início ao fim.

O ponto mais instrutivo do desenvolvimento não foi escrever as classes, e sim descobrir quando o modelo estava errado. A primeira versão tratava cada item do acervo como peça única, o que só se mostrou insuficiente quando o sistema foi exercitado com dados de verdade e a pergunta "quantas cópias deste livro estão disponíveis?" ficou sem resposta. Separar obra de exemplar exigiu alterar o modelo, a persistência e as telas, e deixou claro por que a composição entre as duas classes não é um detalhe de notação, mas uma afirmação sobre o domínio.

Da mesma forma, o tratamento de exceções deixou de ser uma exigência formal e virou a forma natural de comunicar recusas. Uma hierarquia própria permitiu que cada regra violada tivesse a sua exceção, com a mensagem certa, enquanto a superclasse comum manteve a interface gráfica simples, com uma única captura por operação.

Como evolução, o sistema comporta naturalmente a reserva de títulos esgotados, a renovação de empréstimos e a troca da persistência em arquivo por um banco de dados, que afetaria apenas o pacote de persistência, sem tocar no modelo nem nas telas. É um indício de que a separação em camadas cumpriu o seu papel.

REFERÊNCIAS

CAETANO, Roberto. **Notas de aula da disciplina de Programação Orientada a Objetos**. Uberlândia: Faculdade de Computação, Universidade Federal de Uberlândia, 2026.

DEITEL, Paul; DEITEL, Harvey. **Java: como programar**. 10. ed. São Paulo: Pearson, 2017.

ORACLE. **Java Platform, Standard Edition & Java Development Kit Version 21 API Specification**. 2023. Disponível em: https://docs.oracle.com/en/java/javase/21/docs/api/. Acesso em: 4 nov. 2026.

ORACLE. **The Java Tutorials: creating a GUI with Swing**. 2023. Disponível em: https://docs.oracle.com/javase/tutorial/uiswing/. Acesso em: 4 nov. 2026.

SIERRA, Kathy; BATES, Bert. **Use a cabeça! Java**. 2. ed. Rio de Janeiro: Alta Books, 2009.